we find (Section 5) is reasonable for achieving high-quality results, for semantic processing, which is inherently non-exact.

2.3 Core Semantic Operators

We now overview several core semantic operators, providing the operator's definition and at least one gold algorithm for each. We discuss design decisions for our gold algorithms based on state-of-the-art algorithms studied in the AI literature, well-known failure cases, such as long-context challenges [48], and our empirical evaluation in Section 5.

Semantic Filter is a unary operator over the relation T and returns the relation $\{t_i \in T | l_M(t_i) = 1\}$, where the langex provides a natural language predicate over one or more attributes.

Gold Algorithm. Our gold algorithm runs batched LLM calls over all tuples in relation T. Each model invocation, $M(t_i, l)$, prompts the LLM with a single tuple $t_i \in T$, the langex predicate, and an operator-specific instruction to generate a boolean value. This simple choice avoids well-studied long-context issues [48] by processing rows independently rather than batching multiple tuples within a single prompt invocation.

Semantic Join provides a binary operator over relations T_1 and T_2 to return the relation $\{(t_i, t_j) | l_M(t_i, t_j) = 1, t_i \in T_1, t_j \in T_2\}$. Here the langex is parameterized by the left and right join keys and describes a natural language predicate over both.

Gold Algorithm. The gold algorithm implements a *nested-loop join pattern*, performing a single predicate evaluation over a tuple pair with each model invocation $M(t_i, t_j, l) \forall t_i \in T_1, t_j \in T_2$, similar to filters This yields an $O(N_1 \cdot N_2)$ LM call complexity, where N_1 and N_2 are the table sizes of the left and right join tables respectively. This quadratic join algorithm is suitable for small join tables but scales poorly for large tables.

Semanic Top-k imposes a ranking[3] over the relation T and returns the ordered sequence, $\langle t_1, ..., t_k \rangle$ st $\forall (t_i, t_j), i < j \implies l_M(t_i, t_j) = \langle t_i, t_j \rangle$. Here, the langex signature provides a general ranking criteria according to one or more attributes.

Gold Algorithm. Two important algorithmic design decisions for the semantic top-k include how to implement LLM-based comparison and how to aggregate ranking information from these comparisons. Our gold algorithm uses pairwise LLM comparisons for the former, and a quick-select top-k algorithm [33] for the latter. We briefly describe the reason for these choices and alternatives considered. Our decisions build on prior works, which have studied LLM-based passage re-ranking [27, 31, 45, 52, 56–59, 62] and ranking with noisy comparisons [20, 60] with the goal of achieving high quality results in a modest complexity of LLM calls or comparisons.

First, pairwise-prompting methods offer a simple and high-quality approach that feeds a single pair of tuples to each LLM invocation, $M(t_i, t_j, l)$, prompting the model to compare the two inputs and output a binary label. The two main classes of alternatives are point-wise ranking methods [27, 31, 45, 59, 69], and list-wise ranking methods [52, 56, 57, 62], both of which have been shown to face quality issues [27, 58, 62]. We verify these limitations in Section 5. In contrast, pairwise comparisons have been shown to be effective and relatively robust to input ordering [58].

In addition, we consider several possible rank-aggregation algorithms, including quadratic sorting algorithms, a heap-based top-k algorithm and a quick-select-based top-k ranking algorithm. All three alternatives represent candidate gold algorithms. Our evaluation in Section 5 demonstrates that each of these sorting algorithms yield high-quality results, comparable to one another. However, the quick-select-based algorithm offers an efficient implementation with at least an order of magnitude fewer LLM calls than the quadratic sorting algorithm and more opportunities for efficient batched inference, leading to lower execution time, compared to a heap-based implementation. The quick-select top-k algorithm proceeds in successive rounds, each time choosing a pivot, and comparing all other remaining tuples to the pivot tuple to determine the rank of the pivot. Because each round is fully parallelizable, we can efficiently batch these LLM-based comparisons before recursing in the next round.

Semantic Aggregation performs a many-to-one reduce over the input relation, returning $l_M(t_1, ..., t_n), \forall t_1, .., t_n \in T$. Here, the the langex signature provides a commutative and associative aggregation function[4], which can be applied over any subset of rows to produce an intermediate results. We note that the langex iteself is model-agnostic, assuming infinite context. Managing finite context limits of the underlying model M is an implementation detail left to the system.

Gold Algorithm. Our gold algorithm must efficiently orchestrate the LLM to manage long context inputs, which may degrade result quality [48] or overflow the underlying model's context length. We select a hierarchical reduce pattern for the gold algorithm. Our choice builds on the LLM-based summarization pattern studied by prior research works [16, 21, 68] and deployed systems [7, 13], which we briefly overview.

Prior works primarily implement one of two aggregation patterns: either a fold pattern, which performs a sequential, linear pass over the data, while iteratively updating an accumulated partial answer, or a hierarchical reduce pattern, which recursively aggregates the input data to produce partial answers until a single answer remains. Both represent candidate gold algorithms, however, the hierarchical pattern has been shown to produce higher quality results for commutative, associative aggregation tasks, like summarization, in prior work [21] and allows for greater parallelism during query processing, making it our default choice.

Semantic Group-by takes a langex that specifies a projection from a tuple to an unknown group label, as well as a target number of groups, which specifies the desired granularity of group labels. As a example, a user might group-by the topics presented in a set of ArXiv papers, wishing to find 10 key groups. The group-by operator must *discover* representative group labels and assign a label to each each tuple. In general, performing the unsupervised group discovery is a clustering task, which is NP-hard [26]. Clustering algorithms over points in a metric space typically optimize the

[3]This definition implies that l_M imposes a total and consistent ordering. However, this definition can also be softened to assume partial orderings and noisy comparisons with respect to model M.

[4]We note that the ordering of inputs within an LLM prompt invocation can in fact affect results quality for some tasks. Thus, programmers may wish to override the commutativity and associativity assumptions of the semantic aggregation langex. For this, the LOTUS API exposes a partioner function which groups and orders inputs within LLM prompts according to an arbitrary user-specification.

potential function tractably using coordinate descent algorithms, such as k-means. For the semantic group-by, the clustering task is over unstructured fields with a natural language similarity function specified by $l_M(t_i, \mu_j)$, which imposes a real-valued score between a tuple t_i and a candidate label μ_j. This operator poses the following optimization problem:

$$\underset{\{\mu_1,\ldots\mu_C\},\, \mu_i \in V^{\mathbb{N}}}{\arg\max} \sum_{t_i \in T} \max_{j \in 1\ldots C} l_M(t_i, \mu_j)$$

where $\mu_1, \ldots, \mu_C$ are C group labels, each consisting of tokens in the vocabulary, V.

Gold Algorithm. Since this operator, by definition, entails a clustering problem, known to be NP-hard [26], our gold algorithm instead offers a computable implementation to obtain group labels using a tractable clustering algorithm to discover group labels followed by a point-wise classification step. Both steps use a linear LLM pass over the relation.

Specifically, the first stage performs an LLM-based clustering algorithm to discover centers $\mu_1, \ldots, \mu_C$, given user langex. Our gold algorithm first performs a semantic projection, prompting the LLM to predict a label for each input tuple. Then, we embed these candidate labels and perform an efficient vector clustering using a k-means algorithm to construct C groups. For each group, we top-k sample tuples based on centroid-similarity scores, and use a semantic aggregation that prompts the model to generate an appropriate label over each group. In the second stage of our gold algorithm, we use the C generated labels, $\mu_1, \ldots \mu_C$ and invoke the model to perform point-wise assignments $M(t_i, \mu_1, \ldots \mu_C), \forall t_i \in T$.

3 OPTIMIZED EXECUTION PLANS FOR SEMANTIC OPERATORS

In this section, we present novel optimizations for several costly operators, including semantic filters, joins, top-k and group-by. Each optimization provides statistical accuracy guarantees with respect to the gold algorithm, building from Section 2. While this paper focuses on novel optimizations for several expensive semantic operators, we envision a rich space of future work exploring new optimizations and applications of traditional optimizations to semantic operator programs. For example, several prior works demonstrate performance gains in both logical query plan optimizations (e.g. operator re-ordering [46, 47, 50, 51]) and other general LM-approximation techniques (e.g. code synthesis [18, 47] and prompt adaptation [22]).

3.1 Optimizing Semantic Filter

We provide an approximation for semantic filters that obtains recall and precision targets γ_R and γ_P with respect to the gold implementation with probability $1 - \delta$. To achieve reduced cost we leverage the cheaper, but less accurate proxy model $A(t, l)$, which outputs a score indicating whether the tuple t passes the predicate. This idea is inspired by prior works [22, 36, 38, 65, 72, 72], which leverage model cascades for different problem settings. Specifically, several works study cascades in the video analytics setting with vision models, which have significantly different properties than LLMs, requiring different proxy scoring mechanisms. Other works study cascades for LLMs, but use heavy-weight scoring mechanisms to decide whether to use the proxy model and do not provide accuracy guarantees. In contrast, we focus on providing accuracy guarantees for the constrained problem of applying cascades for filters, which allows us to use lighter-weight scoring functions than the more general case studied in prior works.

Algorithm 1: SEM-FILTER(T, l, $M(x)$, $A(x)$, γ_R, γ_P, δ)

Input: Relation T, langex predicate l, oracle model $M(x)$, proxy model $A(x)$, recall target γ_R, precision target γ_P, error probability δ
Output: Filtered relation T'

```
s ← sample_size
S ← ImportanceSample(T, A(x), s)
M_S ← {M(t) : t ∈ S}
A_D ← {A(t) : t ∈ D}
A_S ← {A(t) : t ∈ S}
τ+ ← PT_threshold_estimation(S, l, M_S, A_S, γ_P, δ/2)
τ− ← RT_threshold_estimation(S, l, M_S, A_S, γ_R, δ/2)
τ+ ← max(τ+, τ−)
T' ← ∅
// Evaluate predicate for each tuple
for t ∈ T do
    if A(t) ≥ τ+ then
        T' ← T' ∪ {t}
    else if A(t) ≥ τ− then
        if M(t) then
            T' ← T' ∪ {t}
end
return T'
```

In general, we cannot assume the proxy model is accurate. In fact, the proxy model may perform very poorly for some tasks and records. The goal of our algorithm is to automatically discover the quality of the proxy *at query time* by sampling and comparing to the gold algorithm with the oracle model. Our algorithm will exploit the proxy when it is likely to provide high-quality outputs, and otherwise defer the oracle model. To do this we must learn a decision rule, with learned thresholds on the proxy scores using sampling. As prior work [37] discusses, such sampling-based algorithms introduce multiple-hypothesis testing problems, requiring statistical corrections using confidence intervals, which we carefully apply in our algorithm.

We consider a small LLM as the proxy model and generate scores $A(t)$ using log-probabilities[5] corresponding to the `True` or `False` output tokens of the proxy model's predicate evaluations, re-scaling by the quantiles over all generated log-probabilities over the relation. Algorithm 1 shows the full procedure for performing the approximate semantic filter. We begin by collecting a sample of tuples S, and labeling each sample with the oracle and proxy models. The sampling procedure uses importance sampling and defensively mixes a uniform sample following prior work [38]. Using the central limit theorem and the normal approximation on the distribution of sample statistics, we then learn a decision rule by finding thresholds τ_+ and τ_-, which will respectively ensure the precision target and recall target are met, each with an error probability of $\delta/2$. The statistical sub-procedures to choose each threshold follows prior work [38], but applies them to this new setting where we must ensure *both* targets are met, requiring a correction for hypothesis

[5]This optimizations assumes access to log-probabilities, which is available in common model providers, such as OpenAI, and serving systems, such as vLLM. In the absence of available log-probabilities, alternative uncertainty quantification methods, such as prompt-based methods, may be more suitable.

testing and multiple failure modes. Finally, using the learned decision rule, our algorithm proceeds to process each tuple in the relation. If the tuple's proxy score is at least τ_+, we mark it as passing the predicate. If the tuple's proxy score is less than or equal to τ_-, we mark it as failing the predicate, and for tuples with proxy scores between τ_+ and τ_-, we resort to the oracle model to provide a label.

3.2 Optimizing Semantic Join

Similar to semantic filters, we provide an approximation for semantic joins that obtains recall and precision targets γ_R and γ_P with respect to the gold algorithm with probability $1 - \delta$. Due to the expensive quadratic scaling of the nested-loop join, rather than using a small LLM as the proxy model, we consider an even cheaper proxy based on semantic similarity scores using embeddings. We leverage two possible proxy algorithms and dynamically choose the lowest cost one.

The first approximation, **sim-filter** produces a proxy score $A_1(t_i, t_j)$, $t_i \in T_1, t_j \in T_2$ based on embedding similarity. We perform batched similarity search between the right and left join keys and re-calibrate similarity scores according to their quantiles to obtain proxy scores. This approximation is likely to perform efficiently when tuple pairs with high semantic similarity between the the right and left join key are more likely to pass the predicate. This correlation phenomenon between predicate matches and embedding similarity has been studied by prior works [55] and is not always present. Thus, we introduce an alternative approximation, often suitable when correlation is not present.

The second approximation, **project-sim-filter** first performs a projection over the left join key and then uses the projected column to compute proxy scores $A_2(t'_i, t_j)$ uses the embedding similarity between the projected value t'_i and t_j. As before, we also re-calibrate the proxy scores taking their quantiles. Intuitively, performing the projection is useful when tuple pairs that pass the user's natural language predicate exhibit low semantic similarity. The projection step invokes the LLM over each tuple in left join table, prompting it to predict values for the right join key. Notably, this LLM projection is ungrounded, i.e., it is done without knowledge of the right join key's attribute domain and can thus be performed in a fully parallelized semantic map operation. Figure 3 provides an example of a semantic join between a table of papers and datasets, where the predicate evaluates whether a given paper abstract uses a specific dataset. Here, the map step would invoke the LLM over each abstract, instructing the model to output the dataset used, conditioned only on the abstract.

To dynamically choose between these two approximations, we follow a procedure similar to Algorithm 1 used for semantic filters. We begin by importance sampling to collect the sample S and construct the set of oracle labels, O_S, over the sample. We then obtain tresholds τ_+ and τ_- independently for each approximation algorithm using their respective proxy scores, A_1 and A_2. We use the learned thresholds for each candidate plan to then determine the exact oracle cost needed to execute either algorithm, and we take the least cost plan with its associated learned thresholds to proceed to evaluate the predicate for each tuple pair.

```
papers_df.sem_join(dataset_df, "The paper {abstract:left
    } uses the {dataset:right}.", recall_target=0.9,
    precision_target=0.9, delta=0.1)
```

Figure 3: Example sem_join for matching papers and datasets.

3.3 Optimizing Semantic Group-by

We provide an efficient approximation that guarantees a classification accuracy target, γ, is met with probability $1 - \delta$. We follow the clustering algorithm in the first stage of the gold algorithm to discover centers $\mu_1, ...\mu_C$. We then use a proxy-based approximation for point wise assignments in the second stage of the algorithm. Similar, to semantic joins, we consider semantic similarity scores as a cheap proxy, although other proxy models are also feasible. Specifically, we leverage the embeddings constructed during the clustering stage and compute similarity scores between the the candidate label of each tuple and the discovered centers $\mu_1, ...\mu_C$. The proxy score of the tuple t_i with a predicted label, t'_i, for a discovered center μ_j is given by $A(t_i, \mu_j) = sim(t'_i, \mu_j)$. Our goal is then to learn a threshold, τ, such that if the proxy score between a tuple and center is greater than τ, we return the center as label for the tuple, and otherwise we resort to the more expensive LLM-based classification procedure. We accomplish this by uniform sampling and running the sub-procedure equivalent to the `PT_threshold_estimation` used in Algorithm 1 for semantic filters, where the metric we evaluate is classification accuracy over the C groups.

3.4 Optimizing Semantic Top-k

We leverage the embedding similarity scores to optimize pivot selection for some queries, while incurring no accuracy loss. This optimization is useful when there exists correlation between the ranking imposed by the user's arbitrary sorting criteria and the ranking imposed by semantic similarity scores. In this case, we can sort tuples based on embedding distances to the user's query, and select the $(k + \epsilon)$-th item, rather than a random item, as the first pivot. This can reduce the number of LLM comparisons required by subsequent rounds in the quick-select algorithm, leading to higher query efficiency at no accuracy loss. In the case of no correlation between the langex-based ranking and similarity-based ranking, this method amounts to random pivot selection, and in the worst case of an adversarial pivot, the algorithm will incur one extra pivot round, which can impact execution time, but not degrade quality.

4 THE LOTUS SYSTEMS

In this section we describe the LOTUS system, which implements the semantic operator model as an extension of Pandas [11]. We chose a Pandas-like API in our initial system implementation to make it easy for users to integrate LOTUS with popular AI libraries in Python. However, semantic operators could also be added to a variety of other data processing APIs and query languages, such as SQL. LOTUS leverages vLLM [43] to perform efficient batched inference, and uses FAISS [30, 34] to support efficient vector search for LOTUS' semantic search, similarity join and cluster operations, with indices stored and maintained locally on disk, by default.